UNIVERSITATEA DIN BUCUREȘTI

FACULTATEA DE
MATEMATICĂ ȘI INFORMATICĂ

SPECIALIZAREA TEHNOLOGIA INFORMAȚIEI

Lucrare de licență

PetWell Aplicație pentru managementul animalelor

Absolvent

Bălan Teodor-Daniel

Coordonator științific

Conf.dr. Radu Gramatovici

București, iunie 2025

Rezumat

Creșterea continuă a numărului de animale de companie accentuează nevoia unor soluții informatice care să sprijine gestionarea eficientă a sănătății acestora. În lipsa unei platforme digitale standardizate, procese precum identificarea unui cabinet veterinar, programarea consultațiilor sau păstrarea unui istoric medical coerent rămân adesea fragmentate și ineficiente.

Prezenta lucrare propune dezvoltarea PetWell, o platformă web ce facilitează interacțiunea dintre proprietarii de animale și cabinetele veterinare. Soluția este construită pe un backend Spring Boot și o interfață client React, comunicând printr-un API REST securizat cu JSON Web Tokens. Datele sunt gestionate într-o bază relațională PostgreSQL, iar fișierele clinice (imagini, PDF-uri etc.) sunt stocate în Google Cloud Storage.

Platforma le permite utilizatorilor să consulte istoricul medical al animalelor, să administreze programările și să comunice cu clinicile dintr-un singur loc, accesibil de pe orice dispozitiv. În paralel, medicii veterinari beneficiază de o interfață dedicată pentru gestionarea cazurilor și a documentelor medicale. PetWell își propune astfel să digitalizeze principalele procese administrative și clinice, contribuind la o colaborare mai eficientă între părțile implicate în actul medical veterinar.

Abstract

The continuous increase in the number of companion animals [1] highlights the need for digital solutions that support the efficient management of their health. In the absence of a standardized digital platform, processes such as finding a suitable veterinary clinic, scheduling consultations, or maintaining a coherent medical history often remain fragmented and inefficient.

This thesis proposes the development of PetWell, a web platform designed to facilitate interaction between pet owners and veterinary clinics. The solution is built on a Spring Boot backend and a React client interface, communicating through a REST API secured with JSON Web Tokens. Data is managed in a relational PostgreSQL database, while clinical files (images, PDFs, etc.) are stored in Google Cloud Storage.

The platform allows users to access their pets' medical history, manage appointments, and communicate with clinics from a single, device-independent interface. In parallel, veterinarians benefit from a dedicated interface for managing cases and medical documentation. PetWell thus aims to digitize the core administrative and clinical workflows, contributing to more efficient collaboration among the parties involved in veterinary care.

Cuprins

1 Introducere 5

1.1 Context și motivație 5

1.2 Scopul lucrării 5

1.3 Structura lucrării 6

2 Preliminarii 7

2.1 Soluții digitale deja existente pe piața veterinară 7

3 Arhitectura aplicației 9

3.1 Distribuția funcționalităților pe roluri 9

3.2 Fluxul de date și controlul accesului în aplicație 9

3.3 Diagramă arhitecturală 10

4 Logica aplicației 11

4.1 Motivația alegerii Spring Boot 11

4.2 Gestionarea dependențelor și structura logică 11

4.3 Modelul de date și persistență 12

4.4 Autentificare și autorizare 14

5 Interfața de utilizare 16

5.1 Motivația alegerii React și Mantine 16

5.2 Principii de design 16

5.3 Gestionarea stării și a autentificării 17

5.4 Fluxuri principale de interacțiune 17

5.5 Validarea datelor și feedback vizual 19

5.6 Rutare, protecție și redirecționare 19

6 Livrarea aplicației și infrastructura cloud 20

6.1 Motivația alegerii infrastructurii Docker și EC2 20

6.2 Arhitectura serviciilor definite în docker-compose 20

6.3 Configurația infrastructurii cloud pe AWS 21

7 Concluzii 22

Referințe 23

Anexe 25

A Controller Advice 25

B Postman 26

C Mobile user 27

D Desktop 30

1 Introducere

1.1 Context și motivație

În ultimii ani, creșterea semnificativă a numărului de animale de companie [1] a creat o cerere accelerată pentru servicii veterinare. Totuși, modul în care sunt gestionate informațiile clinice rămâne adesea ineficient și fragmentat. În numeroase cabinete, rezultatele analizelor sunt transmise prin aplicații de mesagerie, prin e-mail sau chiar în format fizic. Deși această practică nu este universală, absența unui sistem informatic integrat rămâne frecvent întâlnită și îngreunează atât activitatea medicilor, cât și accesul proprietarilor la o îngrijire coerentă și transparentă.

Această lipsă de coerență în gestionarea datelor afectează direct calitatea actului medical. Fără acces rapid la istoricul complet al pacientului, medicii veterinari sunt adesea nevoiți să ia decizii pe baza unor informații incomplete sau transmise verbal. În paralel, proprietarii nu dispun de o interfață unificată pentru a urmări programările, tratamentele sau fișierele medicale, ceea ce duce la frustrare și reduce eficiența colaborării.

PetWell se adresează în special cabinetelor veterinare mici și mijlocii, care nu dispun de un sistem informatic propriu, precum și proprietarilor care doresc o evidență digitală clară, centralizată și ușor accesibilă a istoricului medical al animalelor.

1.2 Scopul lucrării

Lucrarea de față își propune să răspundă acestor nevoi prin dezvoltarea și prezentarea platformei PetWell, o soluție software care centralizează datele medicale și administrative dintre cabinetele veterinare și proprietarii de animale. Aplicația oferă o interfață intuitivă, accesibilă din browser, care permite gestionarea fișelor medicale, programărilor și documentelor clinice într-un mod sigur, organizat și eficient.

Din punct de vedere tehnic, platforma este construită cu o arhitectură de tip client-server, cu un backend monolitic implementat în Spring Boot [2], o interfață web realizată în React [3] (SPA), cu o persistență a datelor în PostgreSQL [4] și stocarea fișierelor binare în Google Cloud Storage [5]. Securizarea este realizată prin JSON Web Tokens,

iar întregul ansamblu este containerizat cu Docker [6] și găzduit pe o instanță Amazon Elastic Compute Cloud Documentation [7], folosind Nginx [8] ca reverse proxy și criptare TLS automatizată cu Let's Encrypt [9].

1.3 Structura lucrării

Lucrarea este organizată în șapte capitole principale, fiecare tratând o etapă esențială în dezvoltarea și livrarea aplicației PetWell:

- Capitolul 2 Preliminarii: analiza soluțiilor digitale deja existente pe piața veterinară, evidențiind diferențele față de abordarea propusă de PetWell și prezintă tehnologiile alese.
- Capitolul 3 Arhitectura aplicației: detaliază distribuția funcționalităților în funcție de roluri (utilizator, medic, administrator), modul de control al accesului și structura generală a aplicației, incluzând o diagramă arhitecturală.
- Capitolul 4 Logica aplicației : explică alegerile tehnologice pentru backend (Spring Boot), organizarea codului, modelul de date și mecanismele de autentificare și autorizare utilizate.
- Capitolul 5 Interfața de utilizare: abordează conceptele de design folosite pentru a oferi o experiență unitară atât pe desktop cât și pe mobil, justificând alegerea React și biblioteca Mantine.
- Capitolul 6 Livrarea aplicației și infrastructura cloud: descrie procesul de containerizare cu Docker, organizarea serviciilor în fișierul docker-compose.yml, infrastructura utilizată pe AWS și măsurile de securitate implementate.
- Capitolul 7 Concluzii: sintetizează rezultatele obținute, evidențiază limitările actuale și propune direcții viitoare de extindere pentru platformă.
- Anexe: capturi de ecran pentru interfața aplicației și exemple de apeluri API.

2 Preliminarii

2.1 Soluții digitale deja existente pe piața veterinară

În ultimii ani, au apărut mai multe platforme digitale care urmăresc să eficientizeze activitatea cabinetelor veterinare. Deși oferă funcționalități relevante pentru gestiunea clinicilor, majoritatea acestor soluții sunt construite cu un accent puternic pe nevoile instituționale (stocuri, facturare, programări interne) și mai puțin pe interacțiunea intuitivă cu utilizatorul final proprietarul de animale.

Digitail [10] este o aplicație dezvoltată în România, care oferă o soluție completă de tip PIMS [11]. Include funcții precum fișe clinice structurate, gestiune de stocuri și programări online, alături de un AI assistant care transcrie automat notițele veterinare. Totuși, interfața destinată utilizatorilor este limitată, întrucât aceștia pot efectua programări doar la clinicile în care au fost deja înregistrați de personalul medical, fără posibilitatea de a explora alte cabinete veterinare.

PetDesk [12] adoptă o abordare axată pe retenția clienților și notificări automate (reminders, SMS-uri, e-mailuri), fiind folosită de milioane de utilizatori în America de Nord. Cu toate acestea, platforma nu oferă acces la istoricul medical complet al animalului, iar funcționalitatea aplicației mobile este în mare parte pasivă, reducând rolul proprietarului la unul de simplu beneficiar de notificări.

În contrast cu aceste aplicații, care se concentrează în principal pe optimizarea proceselor interne ale clinicii, PetWell își propune să aducă în prim-plan experiența utilizatorului final proprietarul animalului de companie. Platforma este construită în jurul ideii de transparență, autonomie și accesibilitate digitală, oferind fiecărui utilizator posibilitatea de a vizualiza istoricul medical complet al animalului, de a încărca documente medicale relevante (analize, imagini, rețete) și de a realiza programări direct la clinică, într-un spațiu digital securizat.

Această abordare centrată pe utilizator răspunde unei nevoi reale: majoritatea aplicațiilor existente limitează rolul deținătorului la programări sau notificări pasive, fără a-i oferi control real asupra datelor medicale sau a interacțiunilor cu clinica. PetWell schimbă

acest model, facilitând o colaborare digitală autentică între medic și proprietar.

Alegerea unei arhitecturi monolitice bazate pe Spring Boot permite dezvoltarea rapidă și gestionarea ușoară a codului în faze incipiente, fiind potrivită pentru echipe restrânse și scenarii cu trafic moderat. În plus, expunerea prin API-uri REST securizate cu JWT asigură un echilibru eficient între securitate și simplitate.

Pentru interfață, a fost ales React cu arhitectură de tip SPA (Single Page Application), datorită performanței ridicate în aplicații cu interacțiuni frecvente și nevoii de actualizare rapidă a datelor, fără reîncărcarea completă a paginii. Acest lucru contribuie direct la o experiență de utilizare fluidă și modernă, esențială pentru implicarea utilizatorilor non-tehnici.

La nivel de persistență, baza de date PostgreSQL oferă stabilitate și suport puternic pentru relațiile între entități medicale (animale, fișe, programări), iar Google Cloud Storage este utilizat pentru stocarea fișierelor binare, permițând acces rapid și scalabilitate nativă în cloud.

Pentru livrarea aplicației, utilizarea Docker asigură portabilitate completă, iar rularea pe o instanță Amazon EC2 de tip t3.micro permite costuri reduse și control deplin asupra infrastructurii. Securizarea comunicației se face prin TLS, cu certificate furnizate automat de Let's Encrypt, reducând complexitatea obținerii certificatelor.

În ansamblu, PetWell oferă un cadru tehnic echilibrat, în care funcționalitatea clinicii este dublată de o experiență completă și modernă pentru utilizatorul final ceea ce lipsește majorității soluțiilor comerciale actuale.

3 Arhitectura aplicației

3.1 Distribuția funcționalităților pe roluri

Platforma PetWell este structurată în jurul a trei categorii principale de utilizatori, fiecare beneficiind de funcționalități distincte, adaptate nevoilor specifice ale rolului deținut: proprietari de animale, medici veterinari și administratori de clinică.

Pentru utilizatorii care dețin animale, aplicația oferă posibilitatea de creare a unui cont personal și înregistrare a unuia sau mai multor animale de companie, prin completarea unor formulare. Utilizatorii pot încărca documente medicale (analize, radiografii etc.), consulta istoricul vizitelor, primi notificări automate și localiza clinici în funcție de proximitate și servicii oferite, realizând programări direct din interfață.

Medicii veterinari dispun de o interfață dedicată care include o agendă zilnică interactivă, acces la fișele pacienților și posibilitatea de a actualiza starea fiecărei consultații (planificată, în desfășurare, finalizată). De asemenea, pot atașa fișiere relevante fiecărui caz și accesa istoricul complet al interacțiunilor medicale.

Administratorii de clinică beneficiază de instrumente pentru gestionarea profilului instituției, administrarea personalului și monitorizarea activității. Aceștia pot vizualiza programările tuturor medicilor, redistribui consultații și accesa statistici utile în luarea deciziilor administrative.

Această segmentare a funcționalităților respectă principiul minimului privilegiu și contribuie la protejarea datelor sensibile, oferind totodată o organizare clară a fluxurilor de date și responsabilităților în cadrul aplicației.

3.2 Fluxul de date și controlul accesului în aplicație

Din punct de vedere tehnic, toate cererile clienților sunt rutate prin Nginx, care acționează ca reverse proxy. După autentificare, fiecare utilizator primește un JWT, trimis într-un cookie HttpOnly, semnat HS256 și cu expirație scurtă, ce conține rolul său, backend-ul restricționează accesul la resursele REST în funcție de acest rol. Fișierele binare (PDF-uri, imagini etc.) sunt stocate în Google Cloud Storage, iar metadatele acestora

sunt salvate în PostgreSQL.

Această arhitectură modulară, centrată pe separarea responsabilităților, permite reducerea fragmentării informaționale, optimizarea interacțiunilor dintre utilizatori și oferă o platformă adaptată cerințelor actuale din domeniul veterinar.

3.3 Diagramă arhitecturală

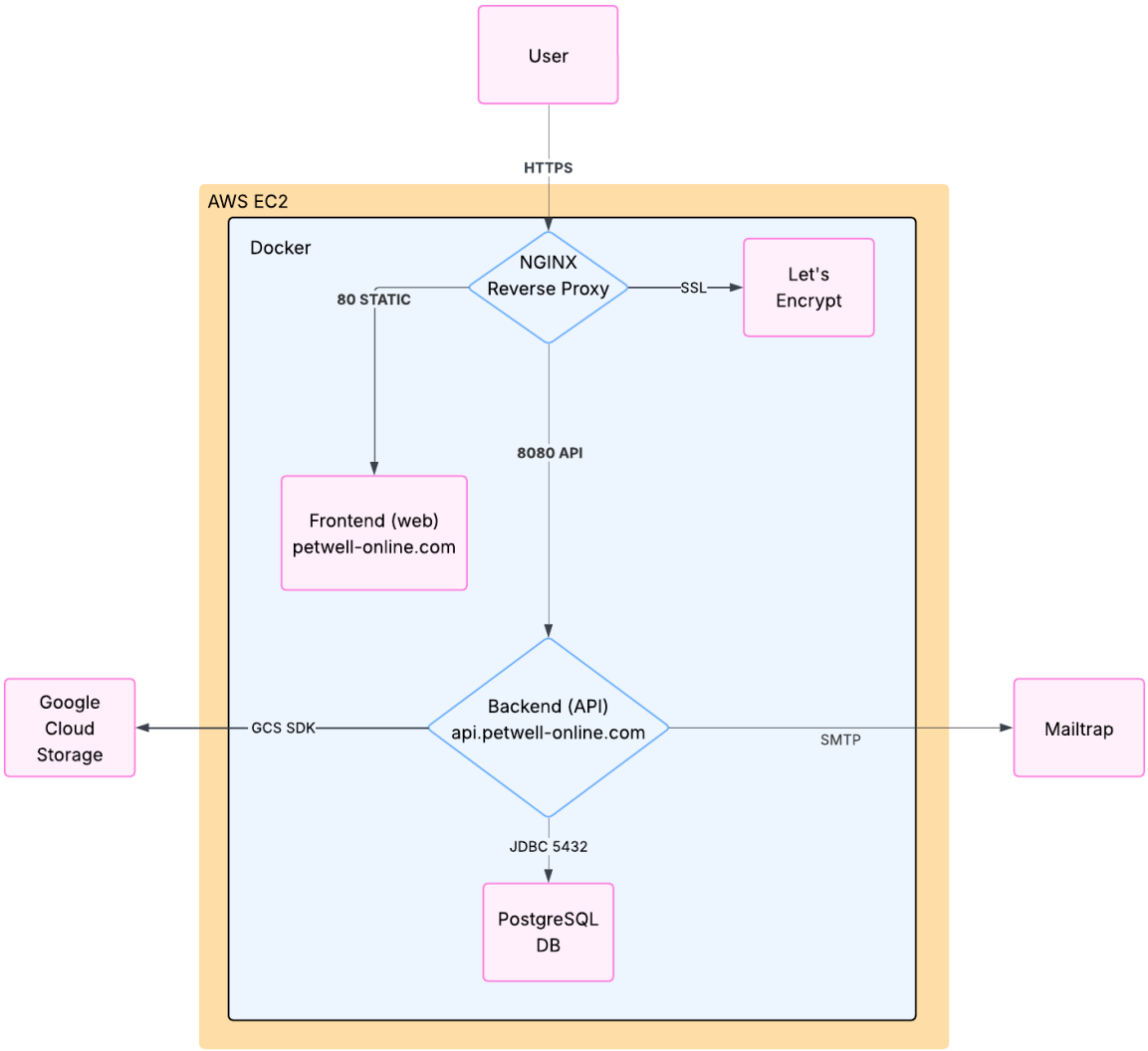


Figura 1: Arhitectura platformei PetWell

În continuare sunt prezentate componentele esențiale platformei PetWell: logica de backend, interfața aplicației și infrastructura de livrare.

4 Logica aplicației

4.1 Motivația alegerii Spring Boot

Pentru dezvoltarea componentei de backend a fost utilizat Spring Boot, o extensie a ecosistemului Spring care permite configurarea rapidă a aplicațiilor Java printr-o structură clară și un sistem eficient de gestionare a dependențelor. Framework-ul oferă suport integrat pentru servicii web, persistenta datelor și securitate, reducând semnificativ efortul de configurare manuală. Alegerea acestuia a fost motivată atât de ușurința integrării cu biblioteci utile proiectului, cât și de compatibilitatea nativă cu Java Virtual Machine. Aplicația poate fi astfel ambalată într-un fișier .jar executabil și containerizată cu ușurință folosind Docker, ceea ce a simplificat procesul de instalare și rulare pe infrastructura aleasă.

4.2 Gestionarea dependențelor și structura logică

Gestionarea dependențelor folosite se realizează prin Maven [13], folosind fișierul pom.xml pentru a include pachetele esențiale: spring-boot-starter-web pentru expunerea endpoint-urilor REST, spring-boot-starter-security cu suport pentru JWT, spring-boot-starter-data-jpa împreună cu driver-ul PostgreSQL pentru persistența relațională, extensia spring-boot-starter-mail configurată pentru trimiterea notificărilor prin SMTP (Mailtrap), precum și biblioteca oficială google-cloud-storage pentru gestionarea fișierelor binare.

Structura codului sursă [14] respectă principiile de separare a responsabilităților, fiind organizată în pachete funcționale. Pachetul model definește entitățile JPA, fiecare reflectând câte o tabelă din baza de date. Accesul la date este abstractizat în pachetul repository, unde interfețele ce extind JpaRepository permit atât interogări declarative, cât și scrierea manuală a codului SQL.

Logica aplicației este structurată în pachetul service, care definește interfețele contractuale pentru operațiile de business. Implementările concrete sunt izolate în service.impl, menținând o separare clară între definirea funcționalității și execuția efectivă. Această arhitectură modulară a facilitat adaptarea codului pe măsură ce proiectul a evoluat, per-

mițând refactorizări punctuale fără a afecta restul aplicației. De exemplu, modificarea tipului de date transmis către bucket-ul din Google Cloud Storage s-a realizat exclusiv în implementarea serviciului responsabil, fără a fi necesară rescrierea interfețelor sau afectarea controllerelor.

Comunicarea cu aplicația are loc prin intermediul controller-elor REST, grupate în pachetul web.controller. Acestea preiau cererile HTTP, apelează serviciile relevante și returnează răspunsuri serializate în format JSON, împărțite corespunzător pe coduri de stare HTTP.

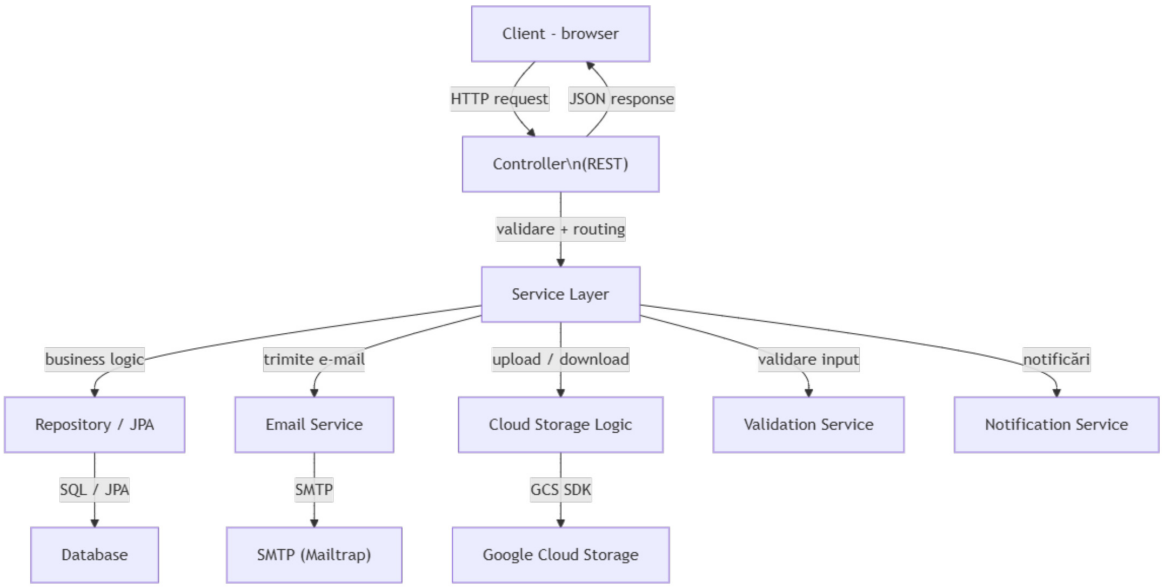


Figura 2: Arhitectura platformei PetWell

Pentru tratarea unitară a excepțiilor, proiectul utilizează un mecanism global bazat pe @ControllerAdvice, care interceptează erorile checked și unchecked și le mapează la coduri HTTP semnificative, oferind astfel clienților un feedback standardizat și predictibil. Anexa A

4.3 Modelul de date și persistență

În centrul platformei PetWell se află o bază de date relațională PostgreSQL, aleasă încă din faza de proiectare pentru a reflecta legăturile multiple și bine definite dintre entități. Aceasta este alcătuită din tabele, fiecare corespunzând unei zone funcționale

distincte a aplicației. Tabela users reunește toate conturile create de persoane fizice și de către proprietarii clinicilor, informațiile aferente cabinetelor veterinare sunt păstrate în veterinary clinics, iar legătura dintre un cabinet și proprietar se realizează prin tabela clinic accounts.

Medicii veterinari sunt gestionați separat, în tabela vets; aceștia nu moștenesc modelul de utilizator, ci își păstrează propriile date de autentificare și profil profesional. Înrolarea lor la una sau mai multe clinici se regăsește în vet clinic memberships, o relație de tip many-to-many care permite, totodată, marcarea rolului ocupat (de exemplu, medic primar sau colaborator extern).

Animalele de companie sunt modelate în tabela pets, cu referință directă către proprietar, iar pentru a avea un istoric ușor accesibil s-a creat tabela one-to-one statistics, ce reprezintă datele necesare pentru a vedea evoluția stării de sănătate a animalului. Programările stocate în tabela appointments leagă un animal, un medic și o clinică, menținând unicitatea perechii (vet, appointment date) pentru a evita suprapunerile din agendă. Rezultatele medicale generate în urma consultațiilor se încarcă în Google Cloud Storage; corespunzător fiecărui fișier, tabela medical records salvează metadatele necesare (nume, cale, checksum) și indică pacientul, medicul sau clinica implicată.

Utilizatorii primesc mail-uri automate ce sunt salvate în tabela email notifications, unde se înregistrează starea fiecărui mesaj transmis. Pentru monitorizare există tabela auxiliară: logs, asociată one-to-one fiecărei clinici și este folosită de către personalul medical pentru asigurarea istoricului și pentru a menține evidența acțiunilor prestate.

Structura bazei de date este realizată prin chei externe, iar regulile ON DELETE CASCADE și SET NULL sunt configurate astfel încât să elimine automat datele dependente sau, după caz, să păstreze istoricul medical relevant. Separarea conținutului binar de stratul relațional optimizează dimensiunea bazei și simplifică scalarea spațiului de stocare în mediul cloud.

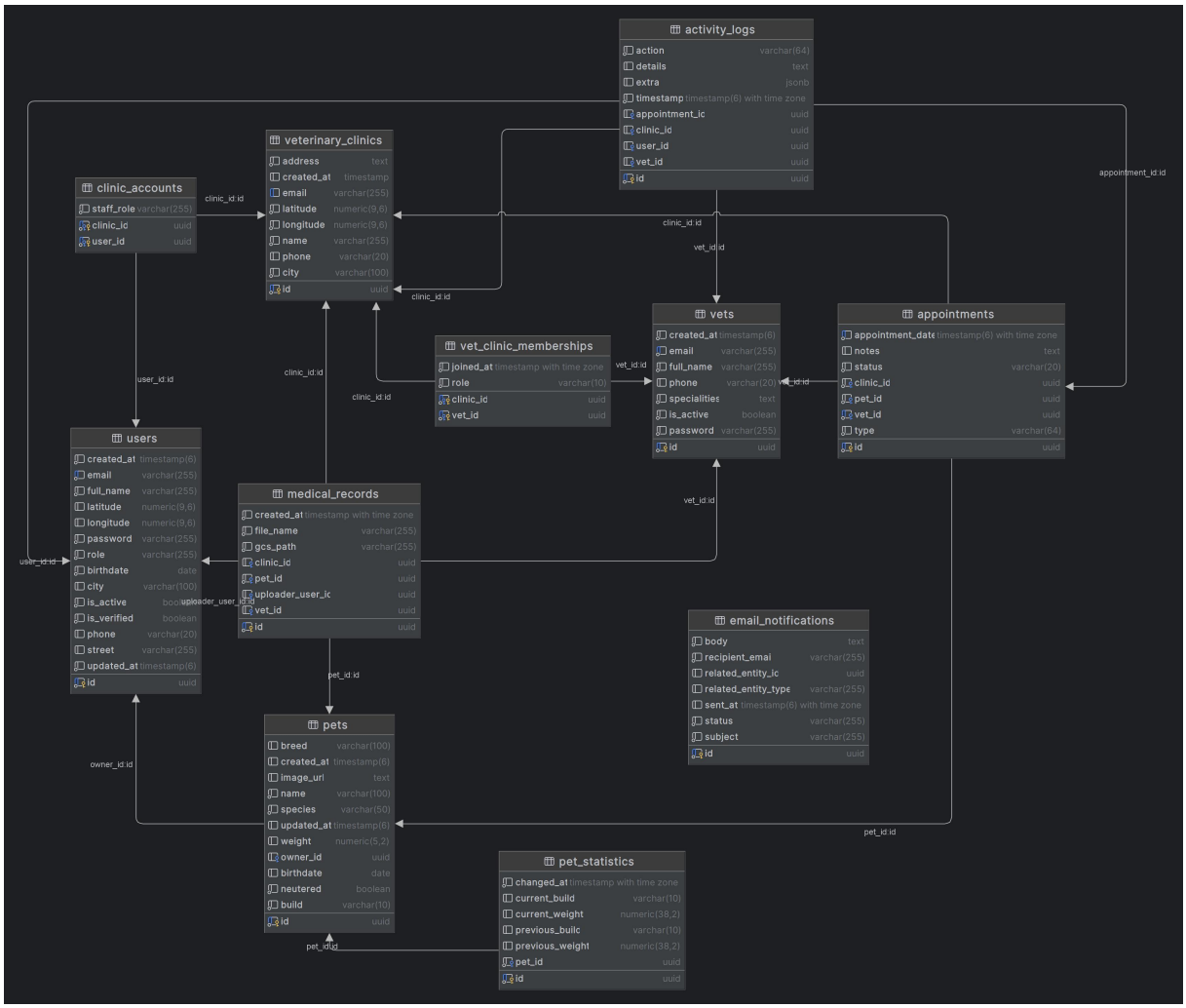


Figura 3: Diagrama Entitate Relație

4.4 Autentificare și autorizare

Accesul la resursele backend este securizat printr-un mecanism de autentificare bazat pe JSON Web Token (JWT). După autentificarea reușită, serverul generează un token semnat, cu expirare scurtă, care include doar identificatorul unic (UUID) al utilizatorului și rolul acestuia. Token-ul este trimis clientului într-un cookie HttpOnly, browserul îl atașează implicit la fiecare cerere, fără a fi expus în JavaScript sau în antetul Authorization. În paralel, backend-ul setează și un cookie refreshToken și limitat la ruta /auth/refresh. Când access-token-ul expiră, endpoint-ul /auth/refresh validează acest cookie și emite un nou JWT, reinjectat prin Set-Cookie, fără intervenția clientului. Filtrul dedicat JwtAuthenticationFilter extrage token-ul din cookie, validează semnătura și expirarea, iar dacă verificările sunt pozitive adaugă în SecurityContext obiectul corespunzător de tip

UserDetails (pentru utilizatori) sau VetDetails (pentru medicii veterinari) Anexa B .

Fișierul SecurityConfig delimitează rutele publice (ex: /auth/login, /auth/register) de cele protejate. Controlul accesului este protejat folosind adnotări @PreAuthorize aplicate direct pe metodele controller-elor, asigurând accesul doar pentru utilizatorii eligibili. De exemplu, metoda getUserById permite doar proprietarului contului să-și vizualizeze datele, iar endpoint-urile administrative (/users) sunt rezervate rolului ADMIN pentru mediul de test.

Medicul veterinar este modelat ca entitate distinctă în tabela vets și nu moștenește structura users, pentru a permite atașarea atributelor profesionale specifice (specializări, program, afiliere multiplă) fără a încărca modelul de bază. Autentificarea ambelor tipuri de utilizatori (USER/OWNER și VET) se realizează prin același mecanism, dar folosind servicii separate: CustomUserDetailsService pentru users, respectiv VetDetailsService pentru vets. Cele două servicii sunt integrate în același flow de login, dar operează pe tabele diferite și aplică reguli specifice de extragere a datelor și rolurilor.

În plus, politica CORS, definită în clasa CorsConfig, permite cereri cross-origin provenite din interfața realizată în React. Expirarea controlată a token-urilor, semnăturile criptografice și extragerea revendicărilor sunt gestionate folosind JwtUtil.

Toate aceste mecanisme din backend susțin o interfață modernă, dezvoltată ca aplicație Single Page Application în React, prezentată în detaliu în capitolul următor

5 Interfața de utilizare

5.1 Motivația alegerii React și Mantine

Interfața platformei PetWell este realizată sub forma unei aplicații de tip Single Page Application, folosind framework-ul React. Această alegere a fost motivată de modelul declarativ de randare, ce actualizea automat interfața în funcție de starea aplicației, precum și de posibilitatea fragmentării codului în componente reutilizabile [15]. Datorită acestei abordări, majoritatea butoanelor și formularelor au putut fi reutilizate în contexte diferite, necesitând doar modificări minore de logică sau stilizare.

Pentru a accelera procesul de dezvoltare și a evita definirea manuală a stilurilor CSS, platforma utilizează biblioteca Mantine UI [16]. Aceasta oferă un set extins de componente predefinite, cu suport nativ pentru responsivitate, validarea formularelor și notificări. Adoptarea acestui design a permis menținerea coerenței vizuale pe întreg parcursul aplicației.

5.2 Principii de design

Aplicația a fost gândită cu un accent pe adaptabilitate, ținând cont de două tipologii principale de utilizatori: proprietarii de animale, care utilizează aplicația predominant de pe smartphone [17], și personalul medical sau administrativ, care accesează funcționalitățile platformei de pe desktop-uri, în cadrul clinicilor.

Pentru utilizatorii non-medicali, experiența este centrată pe utilizarea de pe dispozitive mobile Anexa C. Componentele oferite de Mantine sunt adaptate dinamic prin MediaQuery și sistemul de layout Grid, astfel încât, la o rezoluție sub 768px, interfața este reorganizată vertical. Formularele devin compacte, iar elementele interactive precum butoanele sau câmpurile de input sunt mărite și repoziționate pentru a susține interacțiunea specifică smartphone-urilor.

Astfel componenta care afișează animalele înregistrate: pe varianta desktop, navigarea se face prin butoane cu săgeți poziționate lateral, însă în varianta mobilă, interfața se transformă într-un carusel orizontal controlabil prin gesturi de tip swipe. Această

tranziție asigură o experiență fluidă, intuitivă și adaptată la comportamentul natural al utilizatorilor de pe ecrane mici.

Pentru personalul medical și administrativ, aplicația este concepută pentru ecrane late (peste 1280px), reflectând modul tipic de utilizare în cabinetele veterinare. Dashboard-ul medicilor afișează calendarul consultațiilor și detaliile asociate fiecărui caz într-un layout paralel, ușor de urmărit. În mod similar, interfața de administrare a clinicii include componente tabelare, permițând filtrarea programărilor în funcție de medic, dată sau stare. Aceste mecanisme contribuie la eficientizarea activităților operaționale desfășurate pe desktop Anexa D.

5.3 Gestionarea stării și a autentificării

Starea globală a sesiunii este gestionată printr-un context React dedicat care expune autentificarea, deconectarea și datele utilizatorului curent prin hook-ul reutilizabil useAuth(). La inițializarea aplicației, contextul apelează endpoint-ul /auth/user; serverul validează cookie-ul jwtToken și, dacă token-ul este încă valid, trimite în răspuns doar datele de profil. Browserul atașează cookie-ul automat la fiecare cerere. Dacă validarea eșuează, contextul șterge orice sesiune locală și redirecționează utilizatorul către pagina de login.

Accesul la funcționalitățile aplicației este limitat în funcție de rolul extras din token (OWNER, VET, ADMIN). Componentele sensibile, precum dashboard-ul medicului sau setările de cont, sunt protejate de o componentă PrivateRoute, care blochează randarea în lipsa unei sesiuni active. Astfel, interfața colaborează direct cu backend-ul pentru a aplica restricțiile definite prin adnotările @PreAuthorize, oferind accesul doar pentru utilizatorii autorizați.

5.4 Fluxuri principale de interacțiune

La înregistrare, solicitantul (OWNER, VET sau USER) completează un formular specific rolului ales. După validarea de bază în UI, datele sunt trimise către backend, unde trec prin verificări suplimentare (unicitate e-mail, reguli de parolă, etc.). Dacă cererea

este acceptată (HTTP 200), serverul creează contul și setează, prin antetul Set-Cookie, un JWT semnat care conține doar UUID-ul noului utilizator și rolul acestuia. Cookie-ul este marcat HttpOnly astfel încât browserul îl atașează automat la cererile ulterioare fără să fie accesibil din JavaScript. Front-end-ul cache-ază datele de profil în contextul Auth și redirecționează utilizatorul către pagina principală corespunzătoare rolului său.

Pentru utilizatorii cu rolul USER, interfața principală este inițial aproape goală în absența unui animal înregistrat. Platforma oferă posibilitatea adăugării unui animal fie prin intermediul butonului dedicat „Add Pet", fie din secțiunea „Pets" disponibilă în bara de navigare. După înregistrare, utilizatorul poate atașa analize și documente medicale aferente fiecărui animal. Interfața se actualizează dinamic, afișând noi componente precum: o secțiune cu programările anterioare și curente, un carusel cu animalele înregistrate și acces rapid la fișa detaliată a fiecăruia. În partea inferioară a paginii, utilizatorul poate accesa modulul de localizare, care permite identificarea clinicilor veterinare din proximitate, într-un perimetru configurabil de până la 10 km. Alternativ, în secțiunea „Clinics" sunt disponibile filtre avansate (tipul clinicii, distanță, oraș, servicii oferite), pentru a facilita alegerea unei unități compatibile cu nevoile specifice. După selectarea unei clinici, utilizatorul poate iniția o programare, selectând animalul, medicul dorit, intervalul orar și tipul consultației.

Utilizatorii cu rolul OWNER sunt redirecționați către o interfață dedicată administrării clinicilor. Aceștia pot vizualiza unitățile existente, adăuga o nouă clinică prin completarea unui formular structurat conform modelului de date CLINICS, precum și accesa statistici operaționale. Printre acestea se regăsesc numărul total de programări per clinică, distribuția acestora pe medici și evidențierea celor mai activi angajați, pe baza informațiilor înregistrate în tabela logs.

Pentru utilizatorii cu rolul VET, pagina principală prezintă un calendar interactiv cu programările din ziua curentă. Fiecare consultație poate fi actualizată prin cereri PATCH, modificând starea acesteia (ex.: „în desfășurare", „finalizată") fără reîncărcarea interfeței. De asemenea, medicii pot adăuga fișe medicale pentru animalele consultate și pot accesa istoricul complet al pacienților, arhivat pentru o perioadă de până la zece ani.

Funcționalitatea de localizare a clinicilor este implementată cu ajutorul bibliotecii @react-google-maps/api [18] și este utilizată în două contexte distincte. Pentru utilizatorii USER, modulul permite identificarea clinicilor apropiate în funcție de coordonatele geografice obținute prin API-ul Geolocation, transmise ulterior către endpoint-ul din backend.

Pentru utilizatorii OWNER, componenta este esențială în procesul de înregistrare a unei noi clinici, permițând specificarea exactă a locației acesteia pe hartă. Rezultatele sunt afișate vizual, sub forma unor markere interactive care oferă detalii precum denumirea unității, distanța față de utilizator, intervalul orar disponibil și un buton pentru inițierea unei rezervări rapide. Toate interacțiunile menționate (autentificare, filtrare, completare de date, programare) se realizează asincron, fără reîncărcarea completă a aplicației, oferind o experiență fluidă și coerentă, indiferent de tipul de dispozitiv utilizat.

5.5 Validarea datelor și feedback vizual

Formularele folosesc useForm din biblioteca Mantine, care permite declararea regulilor de validare (câmpuri obligatorii, valori numerice etc.). Erorile sunt afișate în timp real sub câmpurile invalidate, iar feedback-ul general succes sau eroare este comunicat prin notificări vizuale colorate verde (pentru operațiuni reușite) sau roșu (pentru cele eșuate).

5.6 Rutare, protecție și redirecționare

Pentru gestionarea rutelor, aplicația folosește React Router v6 [19]. Rutele publice (precum login sau înregistrare) sunt grupate într-un layout comun, în timp ce rutele protejate sunt înfășurate în componenta PrivateRoute, care verifică dacă utilizatorul este autentificat. În absența unei sesiuni valide, acesta este redirecționat automat către ecranul de login, cu un mesaj tip 403 Access Denied oferit de catre backend.

Această organizare clară a interfeței și a fluxurilor de interacțiune a permis integrarea rapidă a componentelor în infrastructura de producție. În continuare, este prezentat procesul de livrare în producție, incluzând detalii despre containerizare, expunerea aplicației și orchestrarea în mediul cloud.

6 Livrarea aplicației și infrastructura cloud

6.1 Motivația alegerii infrastructurii Docker și EC2

Pentru a asigura un proces de livrare rapid, controlabil și ușor de reprodus, aplicația PetWell a fost containerizată folosind tehnologia Docker și orchestrată local prin intermediul docker-compose [20]. Această abordare permite izolarea clară a componentelor aplicației, facilitând atât testarea în medii locale, cât și implementarea în infrastructuri cloud. Mediul de execuție este găzduit pe o instanță AWS EC2 de tip t3.micro, aleasă datorită includerii în planul free tier oferit de Amazon, dar și pentru integrarea nativă cu alte servicii AWS și posibilitatea configurării rapide a rețelei și a politicilor de acces. Imaginea aplicației este construită local și publicată în Docker Hub, de unde este preluată automat pe serverul EC2 printr-un script.

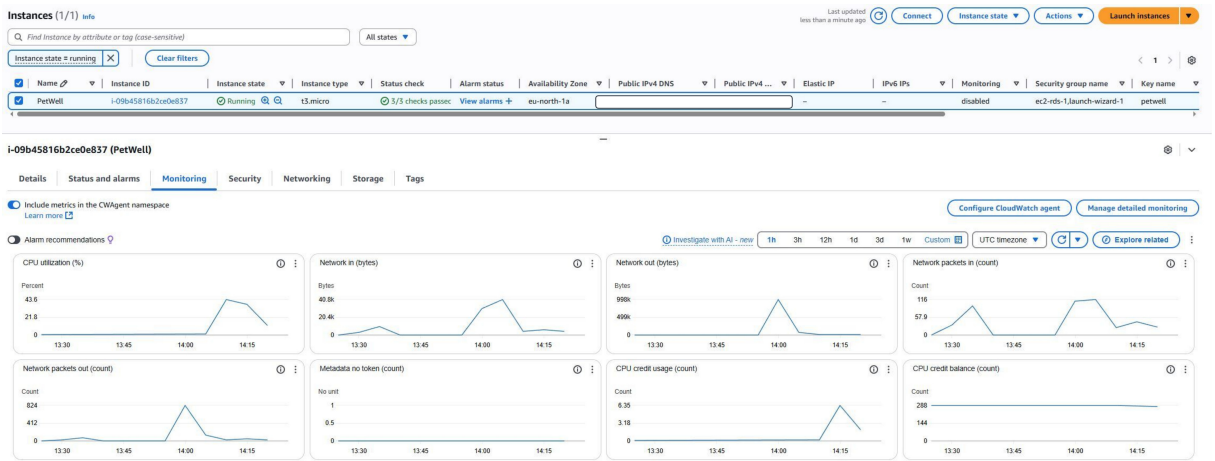


Figura 4: AWS EC2 t3.micro

6.2 Arhitectura serviciilor definite în docker-compose

Fișierul docker-compose.yml [20] definește ansamblul de servicii necesare pentru rularea și expunerea platformei PetWell într-un mediu izolat și scalabil. Arhitectura este compusă din cinci containere principale: reverse-proxy, letsencrypt, frontend, backend și postgres, toate conectate printr-o rețea Docker dedicată (petwell network).

Containerul reverse-proxy funcționează ca punct central de acces pentru întregul trafic HTTP și HTTPS. Acesta termină conexiunile TLS, analizează antetul Host al cererii și

redirecționează traficul către containerul corespunzător, iar containerul letsencrypt automatizează procesul de obținere și reînnoire a certificatelor SSL prin protocolul ACME oferit de Let's Encrypt.

Interfața React se află pe containerul web, construit pe o imagine personalizată publicată în Docker Hub. Backend-ul este reprezentat de o aplicație Spring Boot, conținută în containerul backend, ce preia cereri REST la portul 8080. Ambele containere expun doar porturi interne și sunt accesibile extern doar prin proxy-ul Nginx. Persistența datelor este asigurată de un container PostgreSQL, configurat cu un fișier de inițializare (init.sql) ce definește schema bazei de date.

Pentru toate containere este activată politica restart: always, care permite relansarea automată în cazul unor erori de execuție sau reboot al instanței EC2. De asemenea, volumele montate (certs, html, acme-data) asigură persistența certificatelor SSL și a altor resurse statice gestionate de proxy.

Această arhitectură modulară permite separarea clară a responsabilităților, facilitează mentenanța individuală a serviciilor și asigură scalabilitate ulterioară fără modificări arhitecturale majore.

6.3 Configurația infrastructurii cloud pe AWS

Instanța EC2 utilizată rulează pe un profil t3.micro (1 vCPU, 1 GiB RAM), suficient pentru o aplicație aflată în fază de prototip. Aceasta beneficiază de un IP static obținut prin Elastic IP și este asociată cu domeniile petwell-online.com și api.petwell-online.com prin intermediul serviciului Route 53. Grupul de securitate AWS este configurat restrictiv, permițând acces doar la porturile 22 (SSH), 80 și 443 (HTTP/S), în timp ce porturile interne (5432 pentru PostgreSQL și 8080 pentru backend) sunt vizibile doar în rețeaua Docker.

7 Concluzii

Proiectul prezentat demonstrează că o platformă web modernă, orientată spre digitalizarea serviciilor veterinare, poate fi construită eficient printr-o implementare completă, care acoperă întregul ciclu de dezvoltare software, de la proiectarea modelului de date relațional și definirea API-urilor REST, până la construirea interfeței client și automatizarea procesului de livrare în mediu cloud. PetWell integrează un backend dezvoltat în Spring Boot, cu o arhitectură monolitică modulară, o interfață React construită ca Single Page Application, un model de date în PostgreSQL optimizat pentru relații complexe și un sistem de orchestrare a containerelor prin Docker Compose, rulând pe o instanță EC2 configurată cu proxy revers Nginx, certificat TLS de la Let's Encrypt și DNS gestionat prin AWS Route 53.

Chiar dacă aplicația este complet funcțională, anumite limitări o împiedică să susțină o utilizare la scară extinsă. Baza de date rulează local, fără replicare automată, ceea ce o face vulnerabilă la pierderi de date în cazul unei defecțiuni hardware. Migrarea în AWS RDS, împreună cu replicarea pe mai multe zone de disponibilitate, ar aduce un plus de fiabilitate. De asemenea, în lipsa unui mecanism de autoscalare, extinderea aplicației presupune intervenții manuale. O arhitectură bazată pe ECS Fargate și deploy de tip blue-green ar permite actualizări continue fără întrerupere a serviciului. Observabilitatea este în prezent limitată la loguri și alerte de bază în CloudWatch, iar integrarea completă cu Prometheus și Grafana rămâne un obiectiv viitor.

În concluzie, platforma PetWell oferă un exemplu funcțional de aplicație web construită cu ajutorul tehnologiilor moderne și practici corecte de inginerie software. Proiectul validează ideea că o soluție digitală adaptată domeniului veterinar poate fi implementată cu resurse limitate, dar cu o abordare structurată. Platforma propusă reprezintă o bază solidă pentru extindere ulterioară, atât din punct de vedere funcțional, cât și arhitectural, incluzând perspective precum integrarea cu multiple clinici și laboratoare veterinare.

Referințe

[1] FEDIAF The European Pet Food Industry Federation, https://europeanpetfood.org/about/statistics/, (accesat: 14.06.2025).

[2] Spring Boot, https://docs.spring.io/spring-boot, (accesat: 28.05.2025).

[3] React Documentation, https://react.dev/reference/react, (accesat: 30.05.2025).

[4] PostgreSQL, https://www.postgresql.org/docs/current, (accesat: 25.05.2025).

[5] Google Cloud Storage, https://cloud.google.com/storage/docs, (accesat: 22.05.2025).

[6] Docker, https://docs.docker.com/manuals, (accesat: 23.05.2025).

[7] Amazon Elastic Compute Cloud, https://docs.aws.amazon.com/ec2, (accesat: 14.05.2025).

[8] NGINX, https://docs.nginx.com, (accesat: 14.05.2025).

[9] Let's Encrypt, https://letsencrypt.org/docs, (accesat: 15.04.2025).

[10] DigiTail, https://www.digitail.com, (accesat: 05.06.2025).

[11] The European Data Protection Supervisor, https://www.edps.europa.eu/data-protection/our-work/subjects/personal-information-management-system_en, (accesat: 05.06.2025).

[12] PetDesk, https://petdesk.com/, (accesat: 05.06.2025).

[13] Maven, https://maven.apache.org/pom.html, (accesat: 13.05.2025).

[14] Laurentiu Spilca, Spring Start Here: Learn What You Need and Learn It Well, Manning Publications, 2022, ISBN: 9781617298691.

[15] Mark Tielens Thomas, React in Action, Manning, 2018, ISBN: 9781638356035.

[16] Mantine, https://mantine.dev/, (accesat: 21.05.2025).

[17] StatCounter Global Stats, Desktop vs Mobile Market Share Worldwide, Mai 2025, URL: https://gs.statcounter.com/platform-market-share/desktop-mobile/worldwide, (accesat: 08.06.2025).

[18] Google Maps API, https://developers.google.com/maps/documentation/javascript/tutorials, (accesat: 20.05.2025).

[19] React Router, https://reactrouter.com/home, (accesat: 17.05.2025).

[20] Kathleen Juell, https://www.digitalocean.com/community/tutorials/how-to-secure-a-containerized-node-js-application-with-nginx-let-s-encrypt-and-docker-compose, (accesat: 09.05.2025).

Anexe

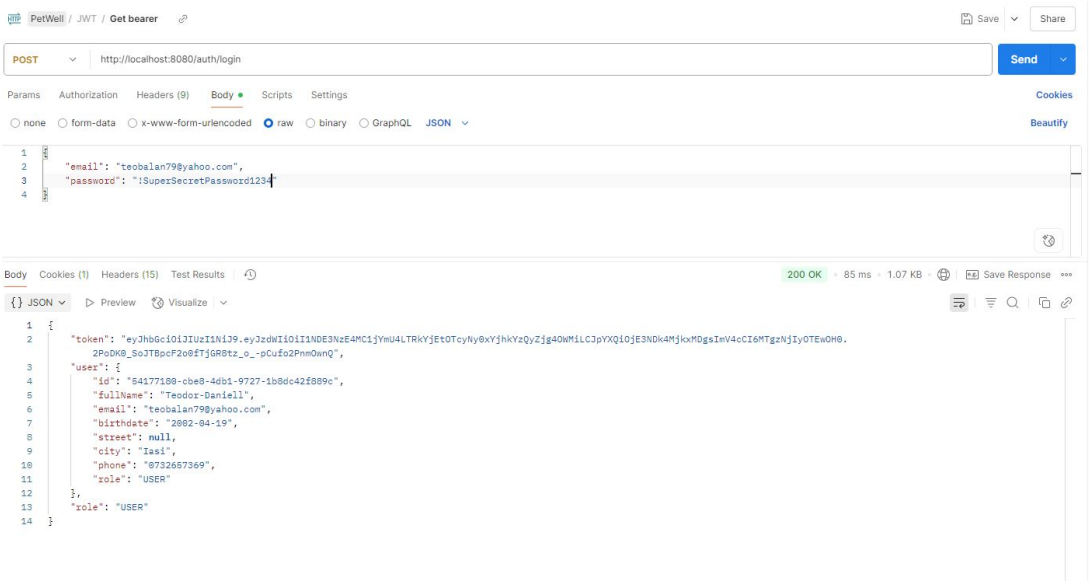
A Controller Advice

```java
@ControllerAdvice  teodor-daniel *
@Slf4j
public class GlobalExceptionHandler {

    // FILE UPLOAD EXCEPTIONS
    @ExceptionHandler(MaxUploadSizeExceededException.class)  teodor-daniel
    public ResponseEntity<ErrorResponse> handleMaxUploadSizeExceeded(MaxUploadSizeExceededException ex) {
        log.warn("File upload size exceeded: {}", ex.getMessage());
        ErrorResponse error = new ErrorResponse(
                errorCode: "FILE_TOO_LARGE",
                message: "File size exceeds maximum allowed limit",
                description: "The uploaded file is too large. Please use a smaller file",
            HttpStatus.PAYLOAD_TOO_LARGE.value()
        );
        return ResponseEntity.status(HttpStatus.PAYLOAD_TOO_LARGE).body(error);
    }

    @ExceptionHandler(StorageException.class)  teodor-daniel
    public ResponseEntity<ErrorResponse> handleStorageException(StorageException ex) {
        log.error("Cloud storage error: {}", ex.getMessage(), ex);
        ErrorResponse error = new ErrorResponse(
                errorCode: "STORAGE_ERROR",
                message: "File storage operation failed",
                description: "Unable to store or retrieve the file from cloud storage",
            HttpStatus.INTERNAL_SERVER_ERROR.value()
        );
        return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR).body(error);
    }

    @ExceptionHandler(IOException.class)  teodor-daniel
    public ResponseEntity<ErrorResponse> handleIOException(IOException ex) {
        log.error("IO error: {}", ex.getMessage(), ex);
        ErrorResponse error = new ErrorResponse(
                errorCode: "IO_ERROR",
                message: "File operation failed",
                description: "An error occurred while processing the file",
            HttpStatus.INTERNAL_SERVER_ERROR.value()
        );
        return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR).body(error);
    }
```

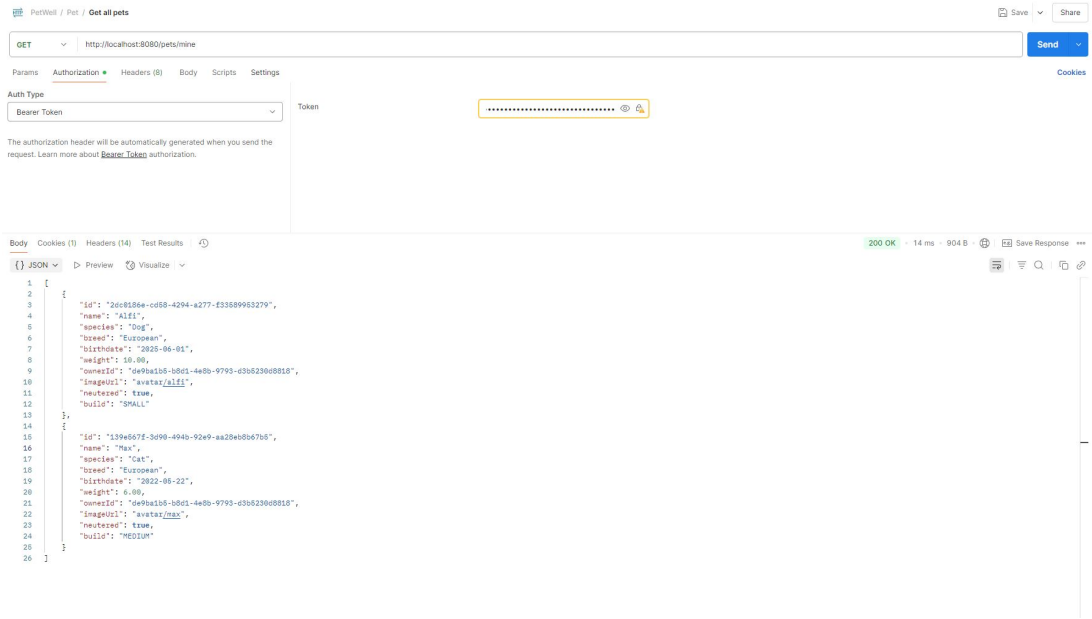
```java
@ExceptionHandler(AccessDeniedException.class)  new *
public ResponseEntity<ErrorResponse> handleAccessDenied(AccessDeniedException ex) {
    log.warn("Access denied: {}", ex.getMessage());
    ErrorResponse error = new ErrorResponse(
            errorCode: "ACCESS_DENIED",
            message: "You are not allowed to access this resource",
            description: "Insufficient permissions to perform the requested operation",
            HttpStatus.FORBIDDEN.value()
    );
    return ResponseEntity.status(HttpStatus.FORBIDDEN).body(error);
}

@ExceptionHandler(BadCredentialsException.class)  new *
public ResponseEntity<ErrorResponse> handleBadCredentials(BadCredentialsException ex) {
    log.warn("Bad credentials provided");
    ErrorResponse error = new ErrorResponse(
            errorCode: "BAD_CREDENTIALS",
            message: "Invalid email or password",
            description: "The provided credentials are incorrect",
            HttpStatus.UNAUTHORIZED.value()
    );
    return ResponseEntity.status(HttpStatus.UNAUTHORIZED).body(error);
}

@ExceptionHandler(AuthenticationException.class)  new *
public ResponseEntity<ErrorResponse> handleAuthenticationException(AuthenticationException ex) {
    log.warn("Authentication failed: {}", ex.getMessage());
    ErrorResponse error = new ErrorResponse(
            errorCode: "AUTHENTICATION_FAILED",
            message: "Authentication failed",
            description: "Unable to authenticate with the provided credentials",
            HttpStatus.UNAUTHORIZED.value()
    );
    return ResponseEntity.status(HttpStatus.UNAUTHORIZED).body(error);
}
```

Figura 5: Exemple de ControllerAdvice PetWell

B Postman



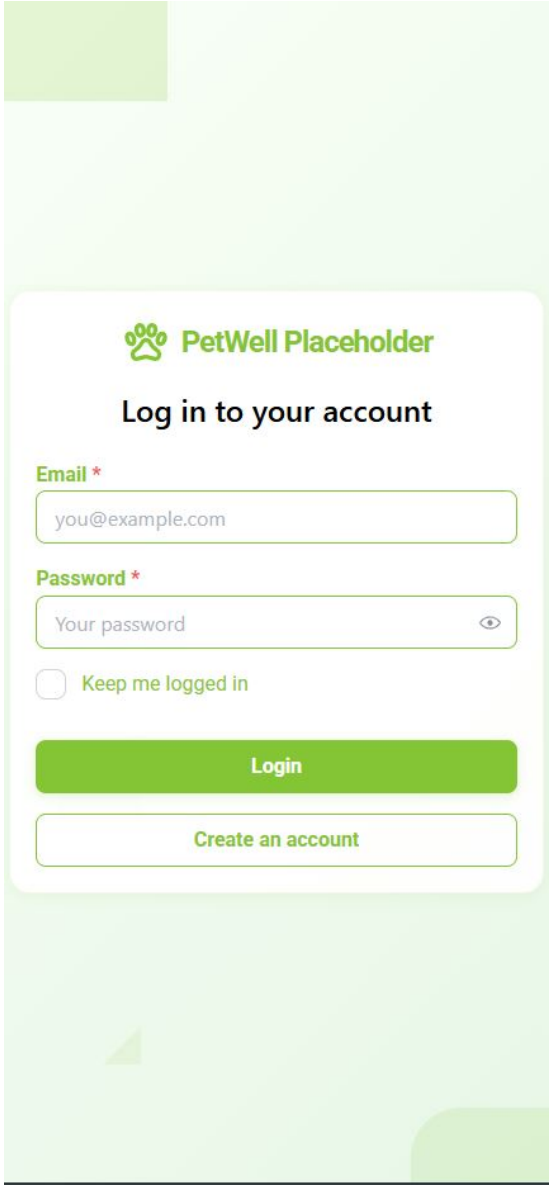
(a) Endpoint login Postman



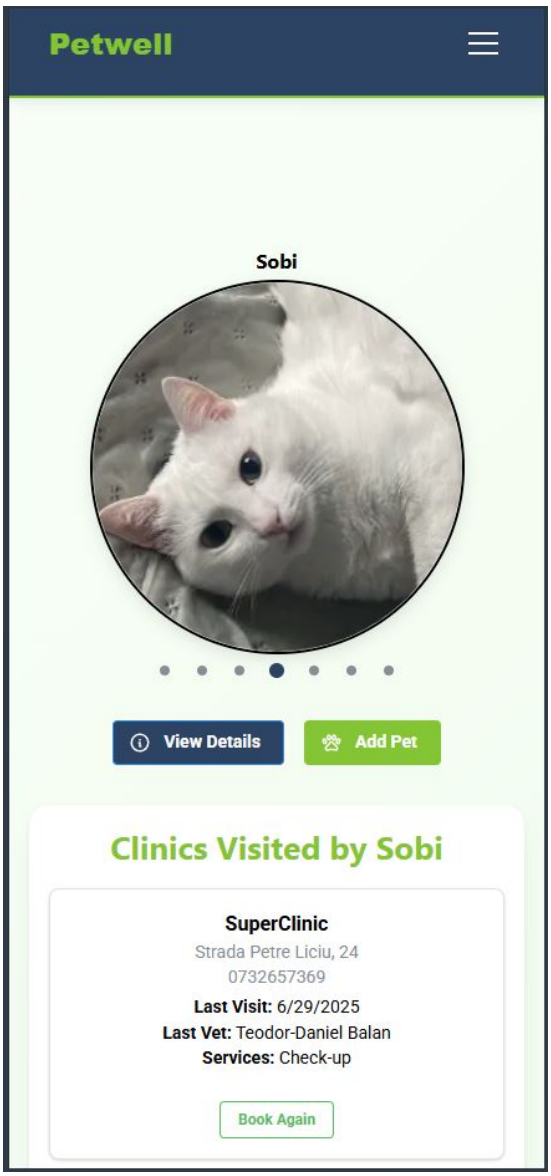
(b) Endpoint pets Postman

Figura 6: Exemple de endpoint-uri în Postman

C Mobile user

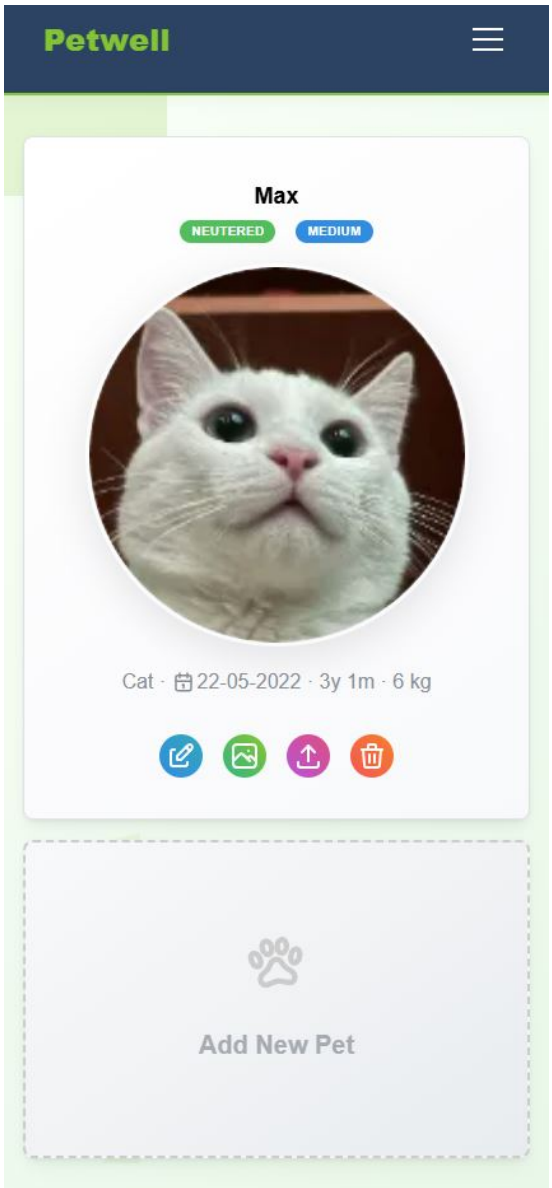


(a) Login mobil

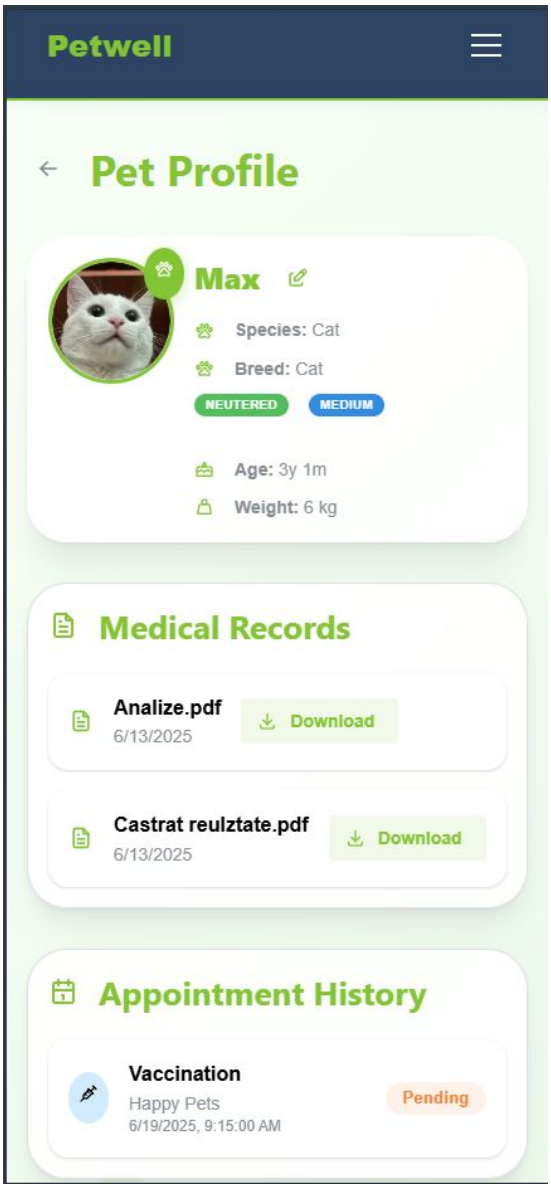


(b) Ecran principal mobil

Figura 7: Pagini principale mobil



(a) Secțiunea Pets mobil



(b) Profil animal mobil

Figura 8: Secțiune animale mobil

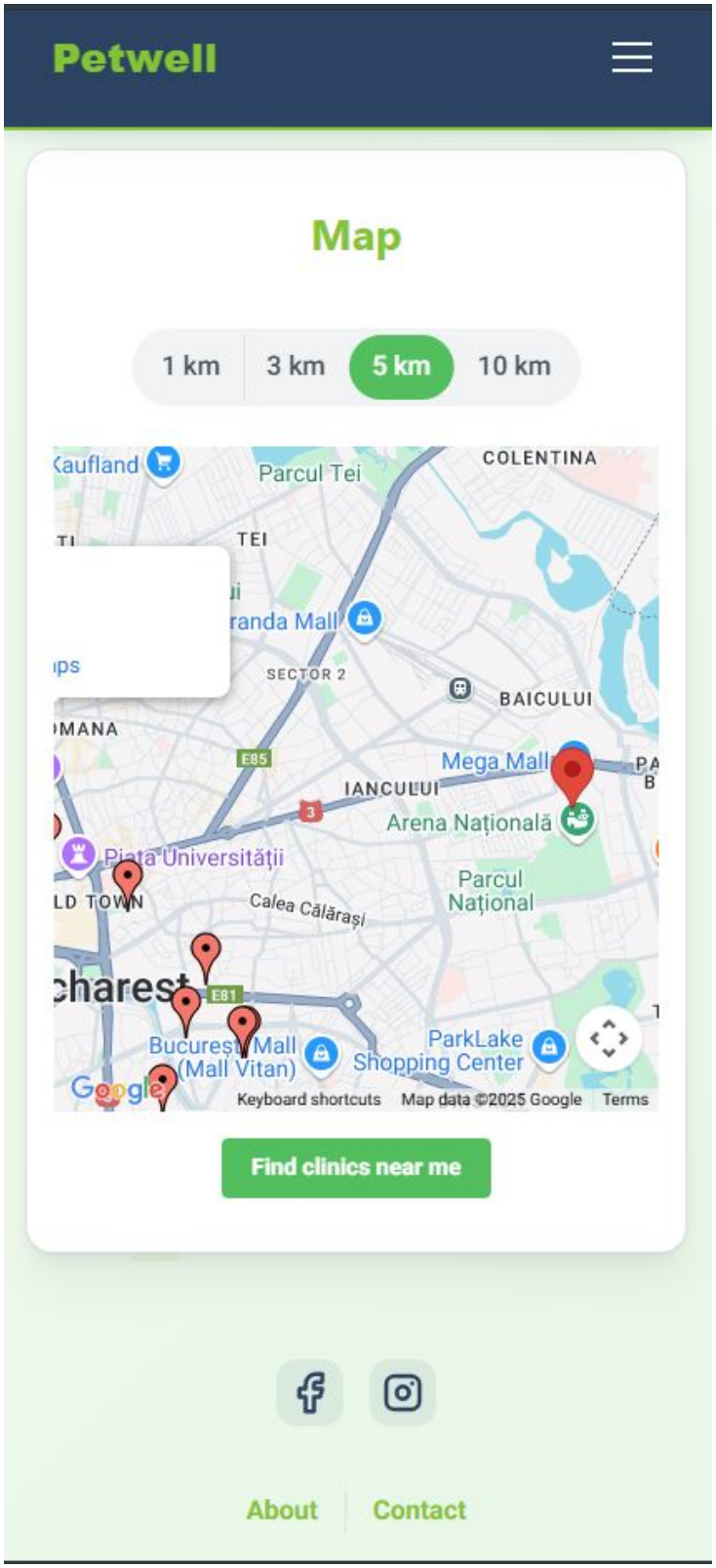


Figura 9: Hartă mobil

D Desktop

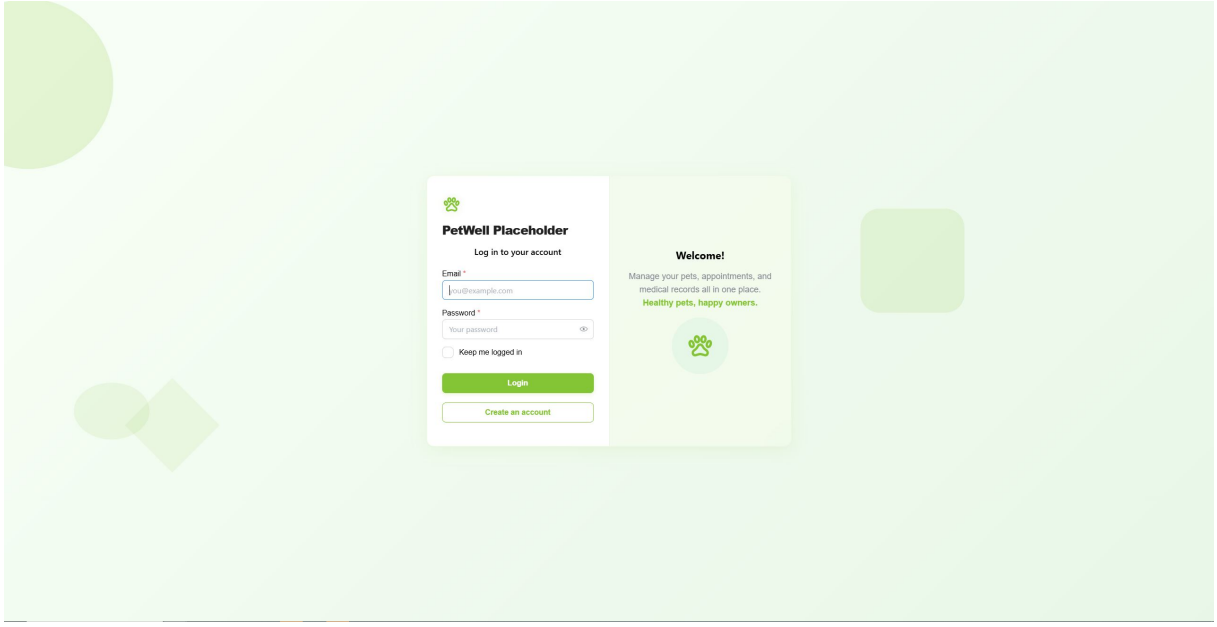


Figura 10: Pagină de autentificare desktop

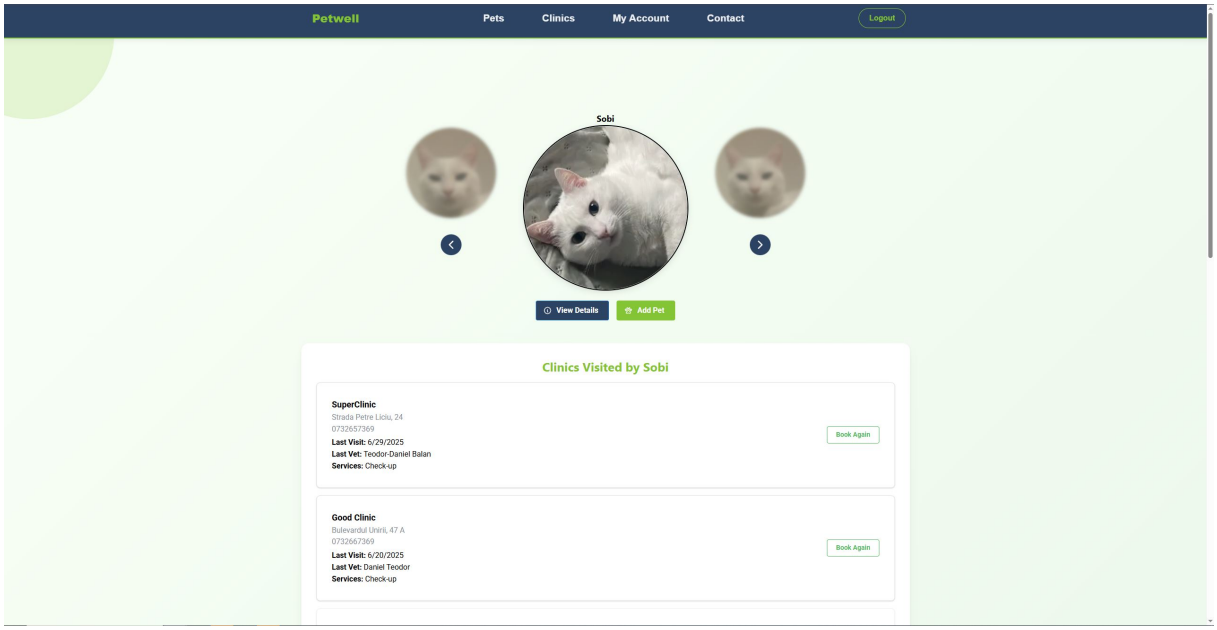


Figura 11: Pagina principală desktop

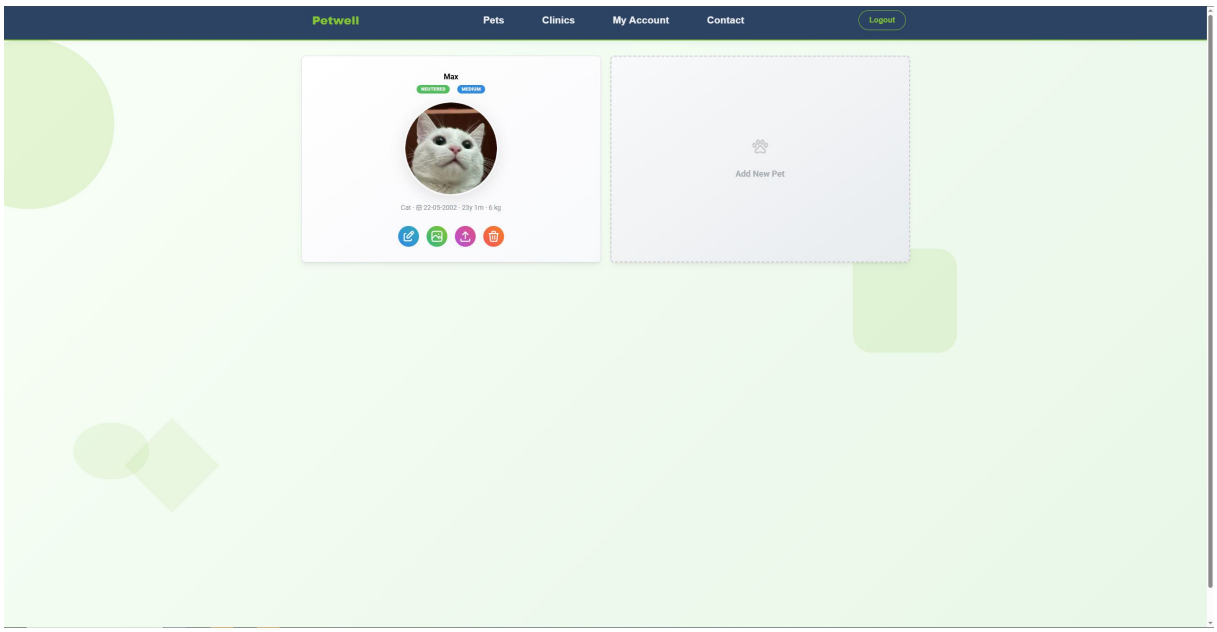


Figura 12: Ecran principal animale de companie desktop

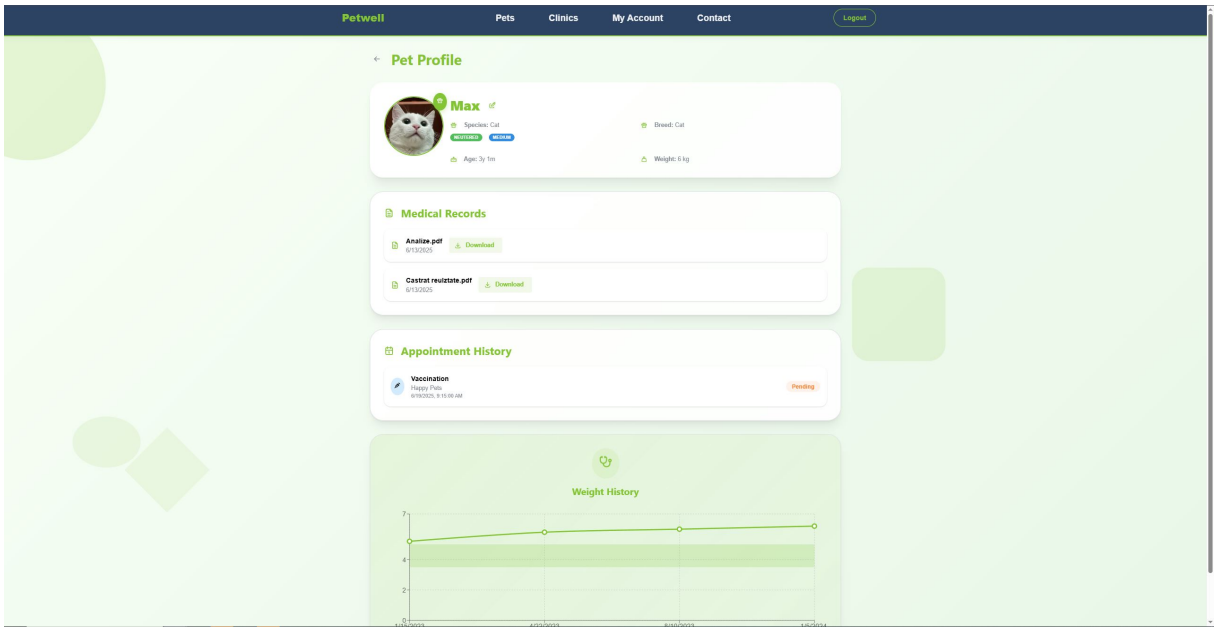


Figura 13: Profil animal de companie desktop

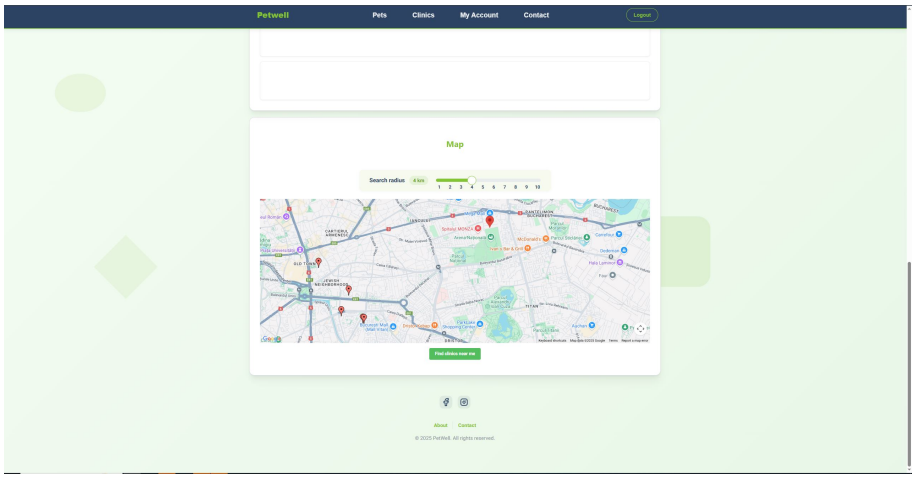


Figura 14: Hartă desktop

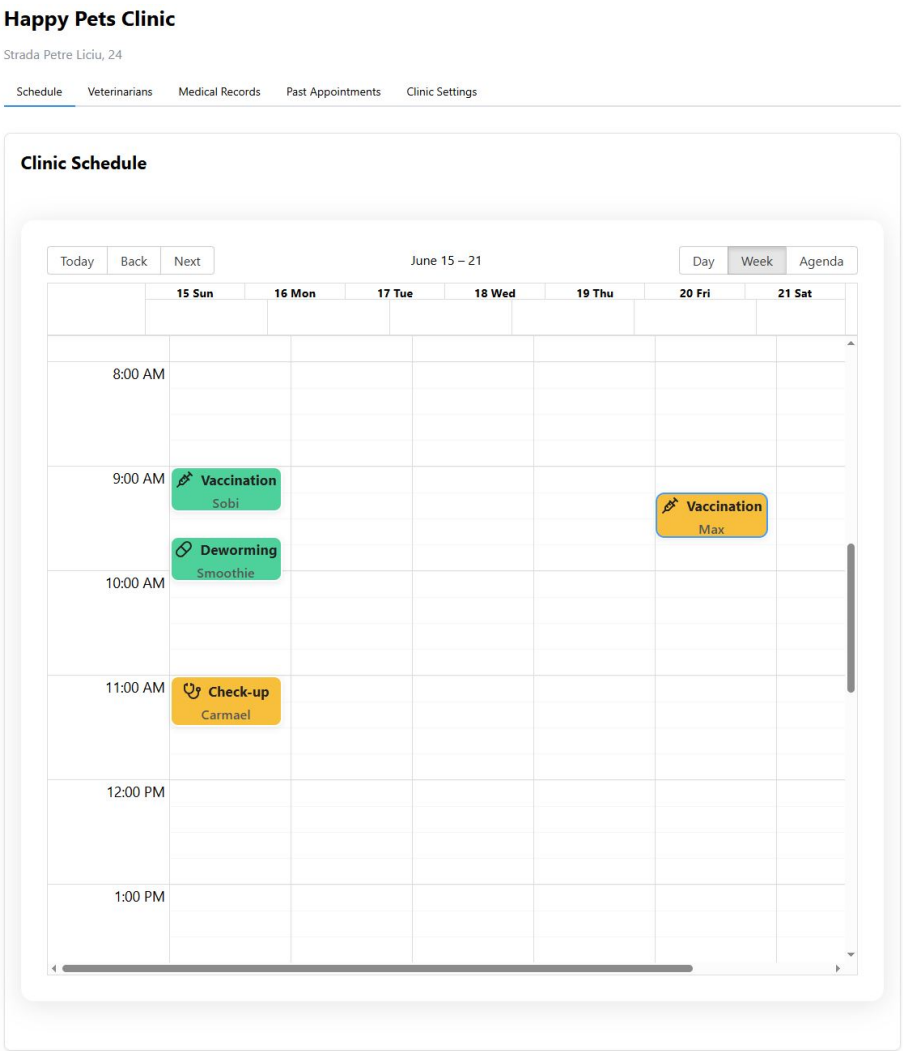


Figura 15: Prezentare clinică desktop